Day 4 of 15 · Core Unit: Prepare Data Sets · LO1

Data Sources, Databases and Your First SQL Queries

Student Handout · TESDA Data Analytics Level III · Batch DAL3-2026

Case company: **Tindahan ni Aling Nena Mini-Mart** · the store's database `aling_nena.db` (branches, products, customers, 500 sales)

Learning Outcomes

By the end of Day 4 you should be able to:

1. Classify **data sources** as **internal or external**, and as **structured, semi-structured or unstructured**.
2. Explain what a **relational database** is: **tables, rows, columns, primary keys and foreign keys**.
3. Open a database in **VS Code with SQLTools** and run a query.
4. Retrieve data with `SELECT`, `FROM` and `DISTINCT`.
5. Filter rows with `WHERE`: `=`, `<>`, `>`, `<`, `AND`, `OR`, `IN`, `BETWEEN`, `LIKE`, `IS NULL`.
6. Sort and shorten results with `ORDER BY` and `LIMIT`.
7. Save your work as a **query script (.sql)**: our evidence for today. *(Integrated: Use information systematically.)*

Materials you need today

Item	File / Location
SQL lab folder (open this folder in VS Code)	`Day04/assets/Day04_SQL_Lab/`
The database	`Day04_SQL_Lab/aling_nena.db`
Your query script (you type your answers here)	`Day04_SQL_Lab/Day04_Queries.sql`
Demo video (captions, shortcut-key overlays)	`Day04/demo/Day04_Demo_Your_First_SQL_Queries.mp4`
Activity sheets	`Day04_Student_Activities.pdf`

> **How to use this handout.** Every new term has a **"Sa simpleng salita"** (in simple words) box. Read that first, then the formal definition. The **Glossary** is on the last page.

Part 1 — Where Does Data Come From? (Data Sources)

> **Sa simpleng salita:** A **data source** is *the place where data comes from*. Aling Nena's cash register is a data source. So is the PAGASA weather forecast, and so is a Facebook comment about her store.

Internal vs external

Type	Meaning	Aling Nena examples
Internal	Data the business creates itself	POS sales receipts, inventory list, suki membership list, employee time records
External	Data that comes from outside the business	PAGASA weather, PSA inflation rates, supplier price lists, Facebook page reviews, GCash settlement reports

Structured, semi-structured, unstructured

> **Sa simpleng salita: Structured** data fits neatly in rows and columns, like a class record. **Unstructured** data does not: photos, voice messages, long comments. **Semi-structured** is in between: it has labels, but not a fixed table.

Type	Looks like	Examples	Easy to analyze?
Structured	Rows and columns with a fixed format	Database tables, Excel sales sheet, CSV file	✅ Yes, directly with SQL or Excel
Semi-structured	Labels/tags, but flexible	JSON from an app, XML, emails (To/From/Subject + free text)	⚠ After some processing
Unstructured	No fixed format	Photos of receipts, voice messages, Facebook comments, CCTV video	❌ Needs special tools

Primary vs secondary data. *Primary* data you collect yourself for your purpose (a customer survey). *Secondary* data was collected by someone else (PSA census figures).

Use information systematically *(Integrated competency)*

Before you pull any data, answer four questions and write them down. This is called a **data source inventory**:

Question	Example
What data do I need?	Daily sales per branch
Where is it (which source/table)?	`aling_nena.db`, table `sales`
Who owns it / who do I ask?	Aling Nena (store owner), branch managers
How good is it (complete, up to date)?	Jan 5 – Mar 29, 2026; 500 sales; walk-in sales have no customer

---

Part 2 — What Is a Relational Database?

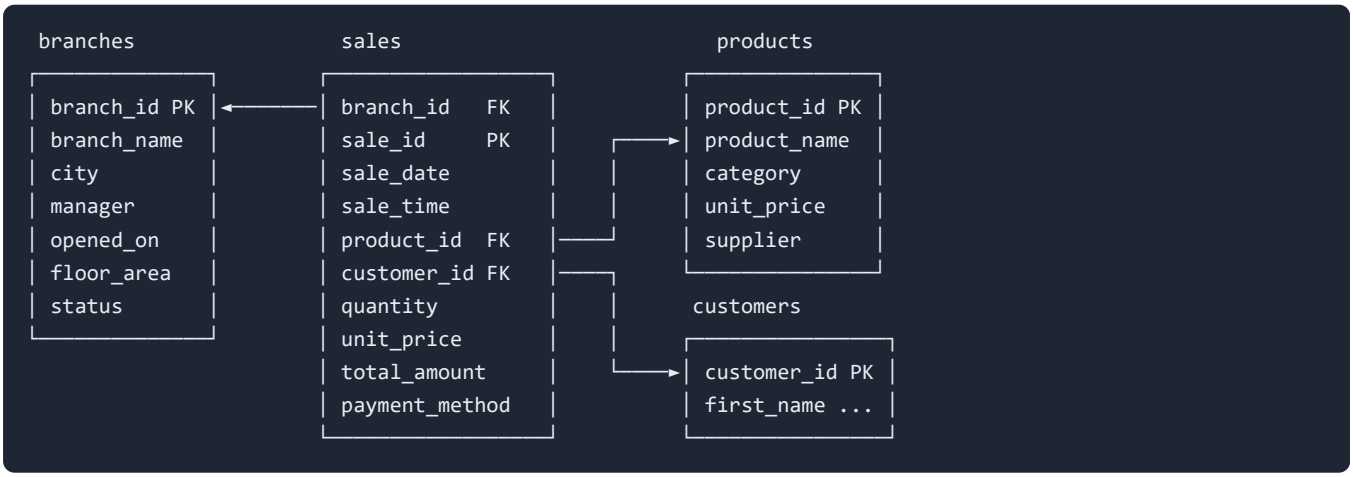
> **Sa simpleng salita:** A **database** is an organized *cabinet* of data. Each **table** is one drawer that holds one kind of thing: one drawer for branches, one for products, one for customers, one for sales. **Relational** means the drawers are *connected* by ID numbers, like a receipt that says "product 107" instead of writing the whole product name again.

Word	Meaning	Example in `aling_nena.db`
Table	A set of rows and columns about one kind of thing	`products`
Row (record)	One item / one event	One product: *Ligo Sardines 155g*
Column (field)	One property, the same type for every row	`unit_price`
Primary key (PK)	A column whose value is **unique** for every row; the row's ID	`product_id` (101, 102, …)
Foreign key (FK)	A column that **points to** the primary key of another table	`sales.product_id` → `products.product_id`
NULL	"No value / unknown". Not zero, not blank text	`sales.customer_id` of a walk-in customer
DBMS	The software that stores and manages the database	SQLite (today), MySQL, PostgreSQL, SQL Server

> **Why not just one big Excel sheet?** If the product name were written on every one of 500 sales, a typo or a price change would have to be fixed 500 times. In a relational database the product is written **once** in `products`, and every sale just points to it. Less repetition, fewer errors.

The Aling Nena database: 4 tables

Table	Rows	Primary key	Foreign keys	What one row means
`branches`	5	`branch_id`	—	One store branch (Antipolo is *Opening Soon*)
`products`	32	`product_id`	—	One product on the shelf
`customers`	46	`customer_id`	—	One **suki** (member) customer
`sales`	500	`sale_id`	`branch_id`, `product_id`, `customer_id`	One line on a receipt (Jan 5 – Mar 29, 2026)



First 3 rows of `sales`:

sale_id	sale_date	sale_time	branch_id	product_id	customer_id	quantity	unit_price	total_amount	payment_method
TXN-10001	2026-01-05	12:37	BR04	107	C040	2	42.0	84.0	GCash
TXN-10002	2026-01-05	13:42	BR02	122	C026	2	28.0	56.0	Debit/Credit Card
TXN-10003	2026-01-05	14:51	BR04	116	*NULL*	1	100.0	100.0	Cash

Read row 3 like this: "*Sale TXN-10003 happened at branch BR04 (Caloocan), for product 116, to a walk-in customer (no customer ID), 1 piece at ₱100, paid in cash.*"

> **Same data as Day 1!** These 500 sales are the same transactions you explored on Day 1 in Excel. That is why SQL will tell you again that **151** sales were paid with GCash, the number from Day 3.

Part 3 — Setting Up: VS Code + SQLTools

> **Sa simpleng salita: SQL** (say "S-Q-L" or "sequel") is the language we use to *ask a database questions*. **VS Code** is the free editor where we type those questions. **SQLTools** is an add-on (extension) for VS Code that sends our questions to the database and shows the answers as a table.

One-time setup on your PC (the trainer has done this on the lab PCs):

1. Install **Node.js LTS** (nodejs.org) and **Visual Studio Code** (code.visualstudio.com).
2. In VS Code press **Ctrl + Shift + X** (Extensions), search **SQLTools**, click **Install**. Then search **SQLTools SQLite** and install it too.

3. The first time you connect, SQLTools may ask to install `sqlite3`. Click **Install now** and wait for "Successfully installed".

Every day: open the lab folder

1. Copy `Day04_SQL_Lab` into your `DAL3/Day04/` folder.
2. In VS Code: **File → Open Folder...** (**Ctrl + K, Ctrl + O**) → choose `Day04_SQL_Lab` → **Select Folder**. If asked "Do you trust the authors?", click **Yes, I trust the authors**.
3. In the Explorer on the left, click **Day04_Queries.sql**.
4. The connection **Aling Nena** is already set up (in `.vscode/settings.json`) and connects by itself. To see it, click the **SQLTools** icon (a database cylinder) in the left bar. Expand **Aling Nena → Tables** to see the 4 tables and their columns.

Run a query

1. Type the query on an empty line. End it with a semicolon `;`.
2. **Select** (highlight) the query: put the cursor at the end of the query, then press **Shift + Home** (one line). For a longer query, press **Shift +** ↑ to select more lines.
3. Press **Ctrl + E**, then **Ctrl + E** again. Or click ▷ **Run on active connection** just above the query.
4. The result opens in a panel below. The bottom-right corner says how many rows, e.g. **1-5 of 5**.
5. Press **Ctrl + S** to save your script often.

Shortcut	What it does
Ctrl + K, Ctrl + O	Open a folder
Ctrl + E, Ctrl + E	Run the selected query (SQLTools)
Shift + Home / Shift + ↑	Select from the cursor to the line start / one more line up
Ctrl + /	Turn the selected lines into comments (`--`) and back
Ctrl + S	Save
Ctrl + Shift + P	Command Palette: type any command, e.g. *SQLTools: Show Table Records*
Ctrl + Z	Undo

Part 4 — SELECT: Choose Your Columns

> **Sa simpleng salita:** `SELECT` = "*show me these columns*". `FROM` = "*from this table*". It's like telling the store helper: "*Show me the names and prices, from the product list.*"

```
SELECT * FROM branches;                                  -- * means every column
SELECT product_name, category, unit_price FROM products;   -- only these 3 columns, in this order
```

Rules for writing SQL

- End every query with `;`.
- Keywords (`SELECT`, `FROM`) can be upper or lower case. We write them in CAPITALS so they are easy to see.
- Column names are separated by **commas**. No comma after the last column.
- Text values go inside **single quotes**: `'GCash'`. Numbers do not: `60`.
- A line starting with `--` is a **comment**: notes for humans that the database ignores.

DISTINCT: each value only once

> **Sa simpleng salita:** `DISTINCT` removes repeats. 500 sales have only 4 payment methods. `DISTINCT` shows each method once.

```
SELECT DISTINCT category FROM products;      -- 12 categories, each once
```

Part 5 — WHERE: Filter the Rows

> **Sa simpleng salita:** `WHERE` = "*only the rows that pass this test*". Like a sieve (salaan): only the rows that match fall through.

```
SELECT product_name, unit_price
FROM products
WHERE category = 'Beverages';
```

Operator	Meaning	Example
`=`	equal to	`WHERE branch_id = 'BR03'`
`<>` (or `!=`)	not equal to	`WHERE payment_method <> 'Cash'`
`>` `<` `>=` `<=`	greater / less than (or equal)	`WHERE unit_price > 60`
`AND`	**both** conditions must be true	`WHERE branch_id = 'BR03' AND payment_method = 'GCash'`
`OR`	**at least one** must be true	`WHERE category = 'Dairy' OR category = 'Bakery'`
`IN (…)`	equal to **any** value in a list (short for many ORs)	`WHERE payment_method IN ('GCash', 'Maya')`
`BETWEEN a AND b`	from a **to** b, **including** both ends	`WHERE sale_date BETWEEN '2026-02-01' AND '2026-02-07'`
`LIKE`	matches a text **pattern**. `%` = any characters, `_` = exactly one character	`WHERE product_name LIKE '%Milk%'`
`IS NULL` / `IS NOT NULL`	has no value / has a value	`WHERE email IS NULL`

> **Watch out!**
> - Dates are stored as text in the form **YYYY-MM-DD** (`'2026-02-14'`), so they sort and compare correctly.
> - **Never write `= NULL`.** It never matches anything. Always write `IS NULL`.
> - Text inside quotes must match **exactly**: `'Gcash'` is not `'GCash'`. (In SQLite, `LIKE` ignores upper/lower case, but `=` does not.)
> - **AND before OR:** `WHERE a OR b AND c` means `a OR (b AND c)`. Use **parentheses** to be clear.

Part 6 — ORDER BY and LIMIT: Sort and Shorten

> **Sa simpleng salita:** `ORDER BY` = "*arrange the rows*": smallest to biggest (**ASC**, the default) or biggest to smallest (**DESC**). `LIMIT 5` = "*only the first 5 rows*". Together they answer "top 5" questions.

```
-- the 5 most expensive products
SELECT product_name, unit_price
FROM products
ORDER BY unit_price DESC
LIMIT 5;
```

product_name	unit_price
Rice - Well-milled (5 kg)	245.0
Spam Classic 340g	210.0
Cooking Oil 1L	110.0
Load - Globe/Smart P100	100.0
Eggs (1 dozen)	96.0

- Sort by **two columns**: `ORDER BY total_amount DESC, sale_id`. The second column breaks ties.
- **The order of the clauses is fixed:**

```
SELECT  columns        -- 1. what to show
FROM    table          -- 2. where from
WHERE   condition      -- 3. which rows
ORDER BY column        -- 4. in what order
LIMIT   n;             -- 5. how many
```

Memory aid: **S**ana **F**riday **W**ill **O**pen **L**ate. **S**ELECT · **F**ROM · **W**HERE · **O**RDER BY · **L**IMIT.

Part 7 — From Business Question to Query

Turn Aling Nena's question into SQL step by step:

> *"Which Pasig sales were paid with GCash? I need the receipt number, date and amount."*

Step	Ask yourself	Answer
1	Which **table**?	`sales`
2	Which **columns** to show?	`sale_id, sale_date, total_amount`
3	Which **rows** (conditions)?	Pasig = `branch_id = 'BR03'` **AND** `payment_method = 'GCash'`
4	Any **order** or **limit**?	Not asked

```
SELECT sale_id, sale_date, total_amount
FROM sales
WHERE branch_id = 'BR03' AND payment_method = 'GCash';
```

Result: **46 rows**. Always read the row count and ask: *does this number make sense?*

Common errors and how to fix them

Message / symptom	Likely cause	Fix
`no such table: product`	Table name misspelled	It is `products` (check the SQLTools tree)
`no such column: price`	Column name misspelled	It is `unit_price`
`near "FROM": syntax error`	Extra comma before `FROM`, or a missing comma between columns	`SELECT a, b FROM …`
`unrecognized token`	A quote was opened but not closed	`'GCash'` needs both quotes
0 rows, no error	Text does not match exactly, or `= NULL`	Check spelling/case; use `IS NULL`
Nothing happens	Query not selected, or not connected	Select it; check the status bar shows *Aling Nena*

Quick Reference

Clause / keyword	Purpose	Example
`SELECT … FROM …`	Show columns from a table	`SELECT * FROM products;`
`DISTINCT`	Remove repeated rows	`SELECT DISTINCT category FROM products;`
`WHERE`	Keep only rows that match	`WHERE unit_price > 60`
`AND` / `OR` / `IN`	Combine conditions	`WHERE payment_method IN ('GCash','Maya') AND quantity >= 3`
`BETWEEN`	A range, both ends included	`WHERE sale_date BETWEEN '2026-02-01' AND '2026-02-07'`
`LIKE`	Text pattern	`WHERE product_name LIKE '%Milk%'`
`IS NULL`	No value	`WHERE email IS NULL`
`ORDER BY … DESC`	Sort, biggest first	`ORDER BY unit_price DESC`
`LIMIT`	First n rows	`LIMIT 5`
`--`	Comment	`-- Q1: all branches`

Glossary

Term	Formal definition	Sa simpleng salita
Data source	The origin from which data is obtained	Kung saan galing ang data
Internal / external data	Data created inside the business / obtained from outside it	Sariling data ng tindahan / galing sa labas
Structured data	Data organized in a fixed schema of rows and columns	Maayos na naka-table
Semi-structured data	Data with tags or labels but no fixed table (JSON, XML)	May label, pero hindi table
Unstructured data	Data with no predefined format (images, audio, free text)	Walang ayos: litrato, boses, comments
Database	An organized collection of data stored electronically	Maayos na lalagyan ng data
Relational database	A database of tables linked by keys	Mga table na magkakakonekta
Table / row / column	A set of records / one record / one attribute	Drawer / isang item / isang katangian

Term	Formal definition	Sa simpleng salita
Primary key (PK)	Column(s) that uniquely identify each row	ID ng bawat row, walang kapareho
Foreign key (FK)	Column that refers to a primary key in another table	Turo sa ID ng ibang table
NULL	The absence of a value	Walang laman, hindi zero
DBMS	Software that manages a database (SQLite, MySQL)	Ang program na nag-aalaga ng database
SQL	Structured Query Language, the standard language for relational databases	Wika para magtanong sa database
Query	A request for data written in SQL	Tanong sa database
Result set	The rows and columns a query returns	Ang sagot na table
Clause	A part of a SQL statement (SELECT, WHERE, ...)	Bahagi ng query

Key Takeaways

- Know your **source** first: internal or external, structured or not, who owns it, how good it is.
- A relational database stores each thing **once**, in its own table, linked by **keys**.
- `SELECT` picks **columns**; `WHERE` picks **rows**; `ORDER BY` sorts; `LIMIT` shortens; `DISTINCT` removes repeats.
- Text in **'single quotes'**, dates as **'YYYY-MM-DD'**, and `IS NULL`, never `= NULL`.
- Always **read the row count** and ask whether it makes sense.

Next session (Day 5): Extract and Combine Data · COUNT, SUM and AVG, aliases, JOINs, and exporting a query result to Excel.